

FC7xxx ETH Integration Manual

Rev.A0

Revision History

Revision	Date	Changes
A0	2025/01/16	Add support for FC7300F8MDQ

Table of Contents

Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area	9
Chapter 5 Interrupt Service Routine (ISR).....	10
Chapter 6 Error Report	11
6.1 Det.....	11
6.2 Dem.....	13
Chapter 7 Function Calls.....	13
7.1 Function Calls during Startup	13
7.2 Function Calls during Shutdown.....	13
7.3 Function Calls during Wake-up.....	13
7.4 Function Calls during Runtime	13
Chapter 8 Other Requirements.....	15
8.1 Notification, Callback, Callout	15
8.2 Macros.....	15
Chapter 9 Integration steps	16

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for ETH module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for Eth module.
- Common: This module is the basic module which used to choose the chip.
- Port: This module provides the IO multiplexing for Eth module.
- ECUC: This module provides the ECUC partition informations for Eth module.
- Dem: This module provides the diagnostic error parameters.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.
- Dem: This module is necessary for reporting diagnostic errors.

Module initialization dependency

- Mcu: To use the Eth module, the user needs to initialize MCU module first.

2.2 Files Required for Compile

ETH module files:

- MCAL/Src/Eth/src/Eth.c
- MCAL/Src/Eth/src/Eth_Hal.c
- MCAL/Src/Eth/ src/Eth_Irq.c
- MCAL/Src/Eth/include/Eth.h
- MCAL/Src/Eth/include/Eth_Hal.h
- MCAL/Src/Eth/include/Eth_Types.h
- MCAL/Src/Eth/include/Eth_Hw.h
- MCAL/Src/Eth/include/Eth_Hw_Types.h
- MCAL/Src/Eth/include/Eth_Reg.h
- MCAL/Src/Eth/include/Eth_Version.h
- _MCAL_generate_8M/src/Eth_PBcfg.c
- _MCAL_generate_8M/include/Eth_Cfg.h

Common module files:

- MCAL/Src/Base/include/Mcal.h
- MCAL/Src/Base/include/Std_Types.h
- MCAL/Src/Base/include/Platform_Types.h
- MCAL/Src/Base/include/Compiler.h
- MCAL/Src/Base/include/Compiler_Cfg.h
- MCAL/Src/Base/include/CompilerDefinition.h
- MCAL/Src/Base/include/ Eth_GeneralTypes.h
- MCAL/Src/Base/include/MemMap/ Eth_MemMap.h
- MCAL/Src/Common/include/ Scm_Reg.h
-

Det module files:

- Det.h
- Det.c

Dem module files:

- Dem.h
- Dem.c

Rte files:

- MCAL/Src/Rte/include/SchM_Eth.h
- MCAL/Src/Rte/include/SchM_Eth.c

2.3 Add Plug-ins

Eth module plug-ins are developed for EB tresos Studio, so, to use Eth plug-ins on the EB tresos Studio, the user needs to add the Eth module to the EB plug-ins folder first.

- 1) Copy the Eth module(MCAL/EB_Plugins/eclipse/plugins/ Eth) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set the Eth module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the Eth module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
ETH_START_SEC_CONFIG_DATA_8 ETH_STOP_SEC_CONFIG_DATA_8 ETH_START_SEC_CONFIG_DATA_16 ETH_STOP_SEC_CONFIG_DATA_16 ETH_START_SEC_CONFIG_DATA_32 ETH_STOP_SEC_CONFIG_DATA_32 ETH_START_SEC_CONFIG_DATA_UNSPECIFIED ETH_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
ETH_START_SEC_CONST_BOOLEAN ETH_STOP_SEC_CONST_BOOLEAN ETH_START_SEC_CONST_8 ETH_STOP_SEC_CONST_8 ETH_START_SEC_CONST_16 ETH_STOP_SEC_CONST_16 ETH_START_SEC_CONST_32 ETH_STOP_SEC_CONST_32 ETH_START_SEC_CONST_UNSPECIFIED ETH_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
ETH_START_SEC_CODE ETH_STOP_SEC_CODE ETH_START_SEC_RAMCODE ETH_STOP_SEC_RAMCODE ETH_START_SEC_CODE_AC ETH_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
ETH_START_SEC_VAR_NO_INIT_BOOLEAN ETH_STOP_SEC_VAR_NO_INIT_BOOLEAN ETH_START_SEC_VAR_NO_INIT_8 ETH_STOP_SEC_VAR_NO_INIT_8 ETH_START_SEC_VAR_NO_INIT_16 ETH_STOP_SEC_VAR_NO_INIT_16 ETH_START_SEC_VAR_NO_INIT_32 ETH_STOP_SEC_VAR_NO_INIT_32 ETH_START_SEC_VAR_NO_INIT_UNSPECIFIED ETH_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
ETH_START_SEC_VAR_INIT_BOOLEAN ETH_STOP_SEC_VAR_INIT_BOOLEAN ETH_START_SEC_VAR_INIT_8 ETH_STOP_SEC_VAR_INIT_8 ETH_START_SEC_VAR_INIT_16 ETH_STOP_SEC_VAR_INIT_16 ETH_START_SEC_VAR_INIT_32 ETH_STOP_SEC_VAR_INIT_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).

ETH_START_SEC_VAR_INIT_UNSPECIFIED ETH_STOP_SEC_VAR_INIT_UNSPECIFIED		
ETH_START_SEC_VAR_NO_INIT_BOOLEAN_NO_CACHEABLE ETH_STOP_SEC_VAR_NO_INIT_BOOLEAN_NO_CACHEABLE ETH_START_SEC_VAR_NO_INIT_8_NO_CACHEABLE ETH_STOP_SEC_VAR_NO_INIT_8_NO_CACHEABLE ETH_START_SEC_VAR_NO_INIT_16_NO_CACHEABLE ETH_STOP_SEC_VAR_NO_INIT_16_NO_CACHEABLE ETH_START_SEC_VAR_NO_INIT_32_NO_CACHEABLE ETH_STOP_SEC_VAR_NO_INIT_32_NO_CACHEABLE ETH_START_SEC_VAR_NO_INIT_UNSPECIFIED_NO_CACHEABLE ETH_STOP_SEC_VAR_NO_INIT_UNSPECIFIED_NO_CACHEABLE	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit, and be placed in noncacheable section. These variables are never initialized by startup code.
ETH_START_SEC_VAR_INIT_BOOLEAN_NO_CACHEABLE ETH_STOP_SEC_VAR_INIT_BOOLEAN_NO_CACHEABLE ETH_START_SEC_VAR_INIT_8_NO_CACHEABLE ETH_STOP_SEC_VAR_INIT_8_NO_CACHEABLE ETH_START_SEC_VAR_INIT_16_NO_CACHEABLE ETH_STOP_SEC_VAR_INIT_16_NO_CACHEABLE ETH_START_SEC_VAR_INIT_32_NO_CACHEABLE ETH_STOP_SEC_VAR_INIT_32_NO_CACHEABLE ETH_START_SEC_VAR_INIT_UNSPECIFIED_NO_CACHEABLE ETH_STOP_SEC_VAR_INIT_UNSPECIFIED_NO_CACHEABLE	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit, and be placed in noncacheable section. These variables are initialized by startup code.

Chapter 4 Exclusive Area

Eth module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the Eth driver:

- Eth_Hal.c:
 - Eth_Hal_AllocTxBuffer: exclusive area 0
 - Eth_Hal_Transmit: exclusive area 1
 - Eth_Hal_ReportTransmission: exclusive area 2
 - Eth_Hal_SetControllerMode: exclusive area 3
 - Eth_Hal_ReportReception: exclusive area 4
 - Eth_Hal_GetCurrentTime: exclusive area 5

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
Eth	ENET_Tx0_IRQHandler	66
	ENET_Tx1_IRQHandler	67
	ENET_Rx0_IRQHandler	68
	ENET_Rx1_IRQHandler	69
	ENET_System_IRQHandler	70

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Eth_Init	ETH_E_PARAM_POINTER ETH_E_ALREADY_INITIALIZED
Eth_SetControllerMode	ETH_E_UNINIT; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_GetControllerMode	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_GetPhysAddr	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_SetPhysAddr	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_UpdatePhysAddrFilter	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_WriteMii	ETH_E_UNINIT; ETH_E_NOT_SUPPORT; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_ReadMii	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_NOT_SUPPORT; ETH_E_INV_PARTITION
Eth_WriteMmd	ETH_E_UNINIT; ETH_E_NOT_SUPPORT; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_ReadMmd	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_NOT_SUPPORT; ETH_E_INV_PARTITION
Eth_GetCounterValues	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX;

	ETH_E_INV_PARTITION
Eth_GetRxStats	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_GetTxStats	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_GetTxErrorCounterValues	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_GetCurrentTime	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_EnableEgressTimeStamp	ETH_E_UNINIT; ETH_E_INV_PARAM; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_GetEgressTimeStamp	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARAM; ETH_E_INV_PARTITION
Eth_GetIngressTimeStamp	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_SetCorrectionTime	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_SetGlobalTime	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_ProvideTxBuffer	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARAM; ETH_E_INV_PARTITION
Eth_Transmit	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARAM; ETH_E_INV_MODE;

	ETH_E_INV_PARTITION
Eth_Receive	ETH_E_UNINIT; ETH_E_PARAM_POINTER; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARAM; ETH_E_INV_MODE; ETH_E_INV_PARTITION
Eth_TxConfirmation	ETH_E_UNINIT; ETH_E_INV_CTRL_IDX; ETH_E_INV_MODE; ETH_E_INV_PARTITION
Eth_TimeAwareShaperInit	ETH_E_UNINIT; ETH_E_INV_CTRL_IDX; ETH_E_INV_PARTITION
Eth_GetVersionInfo	ETH_E_PARAM_POINTER
Eth_MainFunction	ETH_E_UNINIT ETH_E_INV_PARTITION

6.2 Dem

Function Name	Error Type
Eth_Init	ETH_E_ACCESS
Eth_SetControllerMode	ETH_E_ACCESS
Eth_MainFunction	ETH_E_ALIGNMENT; ETH_E_CRC; ETH_E_LATECOLLISION; ETH_E_MULTIPLECOLLISION; ETH_E_OVERSIZEFRAME; ETH_E_RX_FRAMES_LOST; ETH_E_SINGLECOLLISION; ETH_E_UNDERSIZEFRAME
Eth_System_ISR	ETH_E_SAFETY

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API need be called is void Eth_Init(const Eth_ConfigType* CfgPtr).

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

Ethernet SW Driver shall call EthIf_TxConfirmation with Result set to E_OK to indicate a successful transmission; either from the Interrupt routine (in interrupt mode) or from the Eth_TxConfirmation routine in polling mode.

Ethernet SW Driver shall call EthIf_RxIndication to indicate a successful reception either from the Interrupt routine (in interrupt mode) or from the Eth_Receive routine in polling mode.

EthIf_CtrlModeIndication shall be called by Eth_MainFunction when the controller mode changed.

The Eth_WriteMii shall call EthTrcv_WriteMiiIndication when the MII access finished.

The Eth_ReadMii shall call EthTrcv_ReadMiiIndication when the MII access finished.

- Callback

None

- Callout

None

8.2 Macros

Please have a look in various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

AUTOSAR_OS_NOT_USED need defined.

In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example:

```
extern ISR(Eth_System_ISR);

void ENET_System_IRQHandler (void)
{
    Eth_System_ISR ();
}
```

- If OS is used:

Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration steps

- 1) Configure Eth module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the Eth module with dependent modules.